

Experiment 'p_4_5_analyse_floats_seq' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_4_5_analyse_floats_seq

Natural language statement: null

Method signature: p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max: real, range: real)

Ensures

- `average == (sum(values) / |values| as real)`
- `forall v :: v in values ==> min <= v`
- `forall v :: v in values ==> max >= v`
- `range == max - min`
- `exists v :: v in values && min == v`
- `exists v :: v in values && max == v`

Requires

- `0 < |values|`

Functional Code Given

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max: real,
↪ range: real)`

The method should respect the following contract:

`requires 0 < |values|, ensures average == (sum(values) / |values| as real), ensures
↪ forall v :: v in values ==> min <= v, ensures forall v :: v in values ==> max >= v
↪ , ensures range == max - min, ensures exists v :: v in values && min == v, ensures
↪ exists v :: v in values && max == v`

The contract uses the following dafny code:

```
function sum(values: seq<real>): real
{
  if |values| == 0 then 0.0
  else values[0] + sum(values[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnM4R5NQEDMncFxjzNj3Bjo6WLjT8

Model: gpt-5.5-2026-04-23

Created at: 1780655283

Finish reason: stop

Usage: 544 tokens in, and 8453 tokens out

```
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := 0.0;
```

```

assert values[0] in values;
assert values[1..] == [];

forall v: real | v in values
  ensures v == values[0]
{
  var k: int :| 0 <= k < |values| && values[k] == v;
  assert k == 0;
}

assert exists v: real :: v in values && min == v;
assert exists v: real :: v in values && max == v;
} else {
  assert 1 < |values|;
  assert 0 < |values[1..]|;

  var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(values[1..]);
  var tailLen: real := |values[1..]| as real;
  assert tailLen != 0.0;
  assert |values| == 1 + |values[1..]|;

  average := (values[0] + tailAverage * tailLen) / (|values| as real);

  var headIsMin := values[0] <= tailMin;
  if headIsMin {
    min := values[0];
  } else {
    min := tailMin;
  }

  var headIsMax := values[0] >= tailMax;
  if headIsMax {
    max := values[0];
  } else {
    max := tailMax;
  }

  range := max - min;

  assert values[0] in values;

  forall v: real | v in values[1..]
    ensures v in values
  {
    var k: int :| 0 <= k < |values[1..]| && values[1..][k] == v;
    assert 0 <= k + 1 < |values|;
    assert values[1..][k] == values[k + 1];
    assert values[k + 1] == v;
    assert v in values;
  }
}

```

```

forall v: real | v in values
  ensures v == values[0] || v in values[1..]
{
  var k: int :| 0 <= k < |values| && values[k] == v;
  if k == 0 {
    assert v == values[0];
  } else {
    assert 0 <= k - 1 < |values[1..]|;
    assert values[1..][k - 1] == values[k];
    assert values[1..][k - 1] == v;
    assert v in values[1..];
  }
}

```

```

forall v: real | v in values
  ensures min <= v
{
  if v == values[0] {
    if headIsMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert !headIsMin;
    }
  } else {
    assert v in values[1..];
    if headIsMin {
      assert min == values[0];
      assert values[0] <= tailMin;
      assert tailMin <= v;
    } else {
      assert min == tailMin;
      assert tailMin <= v;
    }
  }
}

```

```

forall v: real | v in values
  ensures max >= v
{
  if v == values[0] {
    if headIsMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert !headIsMax;
    }
  } else {
    assert v in values[1..];
    if headIsMax {
      assert max == values[0];
    }
  }
}

```

```

        assert values[0] >= tailMax;
        assert tailMax >= v;
    } else {
        assert max == tailMax;
        assert tailMax >= v;
    }
}

if headIsMin {
    assert min == values[0];
    assert values[0] in values && min == values[0];
    assert exists v: real :: v in values && min == v;
} else {
    assert min == tailMin;
    var w: real :| w in values[1..] && tailMin == w;
    assert w in values;
    assert w in values && min == w;
    assert exists v: real :: v in values && min == v;
}

if headIsMax {
    assert max == values[0];
    assert values[0] in values && max == values[0];
    assert exists v: real :: v in values && max == v;
} else {
    assert max == tailMax;
    var w: real :| w in values[1..] && tailMax == w;
    assert w in values;
    assert w in values && max == w;
    assert exists v: real :: v in values && max == v;
}
}
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_4_5_analyse_floats_seq/program.dfy(15,0): Error: a postcondition could not
  ↳ be proved on this return path
|
15 | {
|   ^

problem_data/p_4_5_analyse_floats_seq/program.dfy(9,17): Related location: this is the
  ↳ postcondition that could not be proved
|
9 | ensures average == (sum(values) / |values| as real)
|   ^^

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
↪ Dafny body, including the curly braces that surround it. Do not show the signature
↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnM7womErwQeXiTlXfm9hnd3JfqTI

Model: gpt-5.5-2026-04-23

Created at: 1780655500

Finish reason: stop

Usage: 1900 tokens in, and 3101 tokens out

```
{
  if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := 0.0;

    assert values[0] in values;
    assert values[1..] == [];
    assert sum(values[1..]) == 0.0;
    assert sum(values) == values[0] + sum(values[1..]);
    assert sum(values) == values[0];
    assert (|values| as real) == 1.0;
    assert average == sum(values) / (|values| as real);

    forall v: real | v in values
      ensures v == values[0]
    {
      var k: int :| 0 <= k < |values| && values[k] == v;
      assert k == 0;
    }

    assert exists v: real :: v in values && min == v;
    assert exists v: real :: v in values && max == v;
  } else {
    assert 1 < |values|;
    assert 0 < |values[1..]|;

    var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(values[1..]);
    var tailLen: real := |values[1..]| as real;
    var len: real := |values| as real;
    assert tailLen > 0.0;
    assert tailLen != 0.0;
    assert |values| == 1 + |values[1..]|;
    assert len == 1.0 + tailLen;
```

```

assert tailAverage == sum(values[1..]) / tailLen;
assert (sum(values[1..]) / tailLen) * tailLen == sum(values[1..]);
assert tailAverage * tailLen == sum(values[1..]);

average := (values[0] + tailAverage * tailLen) / len;

assert sum(values) == values[0] + sum(values[1..]);
calc {
  average;
  ==
  (values[0] + tailAverage * tailLen) / len;
  ==
  (values[0] + sum(values[1..])) / len;
  ==
  sum(values) / len;
  ==
  sum(values) / (|values| as real);
}

var headIsMin := values[0] <= tailMin;
if headIsMin {
  min := values[0];
} else {
  min := tailMin;
}

var headIsMax := values[0] >= tailMax;
if headIsMax {
  max := values[0];
} else {
  max := tailMax;
}

range := max - min;

assert values[0] in values;

forall v: real | v in values[1..]
  ensures v in values
{
  var k: int :| 0 <= k < |values[1..]| && values[1..][k] == v;
  assert 0 <= k + 1 < |values|;
  assert values[1..][k] == values[k + 1];
  assert values[k + 1] == v;
  assert v in values;
}

forall v: real | v in values
  ensures v == values[0] || v in values[1..]
{

```

```

var k: int :| 0 <= k < |values| && values[k] == v;
if k == 0 {
  assert v == values[0];
} else {
  assert 0 <= k - 1 < |values[1..]|;
  assert values[1..][k - 1] == values[k];
  assert values[1..][k - 1] == v;
  assert v in values[1..];
}
}

forall v: real | v in values
  ensures min <= v
{
  if v == values[0] {
    if headIsMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert !(values[0] <= tailMin);
      assert tailMin < values[0];
      assert tailMin <= values[0];
    }
  } else {
    assert v in values[1..];
    if headIsMin {
      assert min == values[0];
      assert values[0] <= tailMin;
      assert tailMin <= v;
    } else {
      assert min == tailMin;
      assert tailMin <= v;
    }
  }
}

forall v: real | v in values
  ensures max >= v
{
  if v == values[0] {
    if headIsMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert !(values[0] >= tailMax);
      assert values[0] < tailMax;
      assert tailMax >= values[0];
    }
  } else {
    assert v in values[1..];
    if headIsMax {

```



```

        assert max == values[0];
        assert values[0] >= tailMax;
        assert tailMax >= v;
    } else {
        assert max == tailMax;
        assert tailMax >= v;
    }
}

if headIsMin {
    assert min == values[0];
    assert values[0] in values && min == values[0];
    assert exists v: real :: v in values && min == v;
} else {
    assert min == tailMin;
    var w: real :| w in values[1..] && tailMin == w;
    assert w in values;
    assert w in values && min == w;
    assert exists v: real :: v in values && min == v;
}

if headIsMax {
    assert max == values[0];
    assert values[0] in values && max == values[0];
    assert exists v: real :: v in values && max == v;
} else {
    assert max == tailMax;
    var w: real :| w in values[1..] && tailMax == w;
    assert w in values;
    assert w in values && max == w;
    assert exists v: real :: v in values && max == v;
}
}
}

```

Final Program

```

function sum(values: seq<real>): real
{
    if |values| == 0 then 0.0
    else values[0] + sum(values[1..])
}

method p_4_5_analyse_floats_seq(values: seq<real>) returns (average: real, min: real, max
    ↪ : real, range: real)
    requires 0 < |values|
    ensures average == (sum(values) / |values| as real)
    ensures forall v :: v in values ==> min <= v
    ensures forall v :: v in values ==> max >= v

```

```

    ensures range == max - min
    ensures exists v :: v in values && min == v
    ensures exists v :: v in values && max == v
{
if |values| == 1 {
    average := values[0];
    min := values[0];
    max := values[0];
    range := 0.0;

    assert values[0] in values;
    assert values[1..] == [];
    assert sum(values[1..]) == 0.0;
    assert sum(values) == values[0] + sum(values[1..]);
    assert sum(values) == values[0];
    assert (|values| as real) == 1.0;
    assert average == sum(values) / (|values| as real);

    forall v: real | v in values
        ensures v == values[0]
    {
        var k: int :| 0 <= k < |values| && values[k] == v;
        assert k == 0;
    }

    assert exists v: real :: v in values && min == v;
    assert exists v: real :: v in values && max == v;
} else {
    assert 1 < |values|;
    assert 0 < |values[1..]|;

    var tailAverage, tailMin, tailMax, tailRange := p_4_5_analyse_floats_seq(values[1..]);
    var tailLen: real := |values[1..]| as real;
    var len: real := |values| as real;
    assert tailLen > 0.0;
    assert tailLen != 0.0;
    assert |values| == 1 + |values[1..]|;
    assert len == 1.0 + tailLen;

    assert tailAverage == sum(values[1..]) / tailLen;
    assert (sum(values[1..]) / tailLen) * tailLen == sum(values[1..]);
    assert tailAverage * tailLen == sum(values[1..]);

    average := (values[0] + tailAverage * tailLen) / len;

    assert sum(values) == values[0] + sum(values[1..]);
    calc {
        average;
        ==
        (values[0] + tailAverage * tailLen) / len;
        ==

```

```

    (values[0] + sum(values[1..])) / len;
  ==
  sum(values) / len;
  ==
  sum(values) / (|values| as real);
}

var headIsMin := values[0] <= tailMin;
if headIsMin {
  min := values[0];
} else {
  min := tailMin;
}

var headIsMax := values[0] >= tailMax;
if headIsMax {
  max := values[0];
} else {
  max := tailMax;
}

range := max - min;

assert values[0] in values;

forall v: real | v in values[1..]
  ensures v in values
{
  var k: int :| 0 <= k < |values[1..]| && values[1..][k] == v;
  assert 0 <= k + 1 < |values|;
  assert values[1..][k] == values[k + 1];
  assert values[k + 1] == v;
  assert v in values;
}

forall v: real | v in values
  ensures v == values[0] || v in values[1..]
{
  var k: int :| 0 <= k < |values| && values[k] == v;
  if k == 0 {
    assert v == values[0];
  } else {
    assert 0 <= k - 1 < |values[1..]|;
    assert values[1..][k - 1] == values[k];
    assert values[1..][k - 1] == v;
    assert v in values[1..];
  }
}

forall v: real | v in values
  ensures min <= v

```

```

{
  if v == values[0] {
    if headIsMin {
      assert min == values[0];
    } else {
      assert min == tailMin;
      assert !(values[0] <= tailMin);
      assert tailMin < values[0];
      assert tailMin <= values[0];
    }
  } else {
    assert v in values[1..];
    if headIsMin {
      assert min == values[0];
      assert values[0] <= tailMin;
      assert tailMin <= v;
    } else {
      assert min == tailMin;
      assert tailMin <= v;
    }
  }
}

forall v: real | v in values
  ensures max >= v
{
  if v == values[0] {
    if headIsMax {
      assert max == values[0];
    } else {
      assert max == tailMax;
      assert !(values[0] >= tailMax);
      assert values[0] < tailMax;
      assert tailMax >= values[0];
    }
  } else {
    assert v in values[1..];
    if headIsMax {
      assert max == values[0];
      assert values[0] >= tailMax;
      assert tailMax >= v;
    } else {
      assert max == tailMax;
      assert tailMax >= v;
    }
  }
}

if headIsMin {
  assert min == values[0];
  assert values[0] in values && min == values[0];
}

```

```

    assert exists v: real :: v in values && min == v;
  } else {
    assert min == tailMin;
    var w: real :| w in values[1..] && tailMin == w;
    assert w in values;
    assert w in values && min == w;
    assert exists v: real :: v in values && min == v;
  }

  if headIsMax {
    assert max == values[0];
    assert values[0] in values && max == values[0];
    assert exists v: real :: v in values && max == v;
  } else {
    assert max == tailMax;
    var w: real :| w in values[1..] && tailMax == w;
    assert w in values;
    assert w in values && max == w;
    assert exists v: real :: v in values && max == v;
  }
}
}

```

Total Token Usage

Input tokens: 2444

Output tokens: 11554

Reasoning tokens: 8677

Sum of 'total tokens': 13998

Experiment Timings

Overall Experiment started at 1780655281597, ended at 1780655574700, lasting 293103ms (293.10 seconds)

Iteration #1 started at 1780655281664, ended at 1780655499280, lasting 217616ms (217.62 seconds)

Iteration #2 started at 1780655499280, ended at 1780655574700, lasting 75420ms (75.42 seconds)